IT645- Course Project Guidelines

The project should be developed using the MERN stack. Students must present a working demonstration of the project, including the frontend, backend, and database. Each group member must clearly explain their individual contribution and the relevant code.

Note that the project scope should be as per the team size to allow each member to make a substantial contribution. A rule of thumb is to have at least one major functional module per team member where each member must make a substantial contribution to frontend, backend and testing of the project. Your team should not work in such a way that a student completely develops frontend and another backend, while a third is doing testing.

Evaluation of each member will be done independently based on his/her own presentation/demonstration of his contribution and viva. Based on answers of a couple of students, rest will not be awarded marks.

Evaluation will be conducted as per the following guidelines.

A. Working Application & Functionality

a. Demonstrate complete frontend → backend → database integration.
b. Demonstrate the complete working flow

B. Frontend (HTML, CSS, JS and React)

a. Use semantic HTML
b. Your application should support responsive web design.
c. Use Tailwind CSS appropriately.
d. Develop the UI using React functional components.
e. Use React hooks appropriately, including useState and useEffect; demonstrate other hooks such as useRef/useContext where relevant.
f. Use React Router for navigation.
g. Use fetch() (or axios) to communicate using HTTP requests with the backend.
h. Handle loading, success, and error states.
i. Use at least one meaningful third-party Web API.

C. Backend (Node, ExpressJS and MongoDB)

Develop RESTful APIs using Node.js and Express.js with following:

a. Structure URLs with clear endpoints (e.g., /api/users/:id) where each API endpoint should represent a unique resource. Use route parameters to identify a resource. Use query parameters appropriately for operations such as filtering, searching, sorting, or pagination.
b. Use all HTTP methods GET POST PUT DELETE PATCH atleast once
c. Implement proper status codes for responses (e.g., 200 for success, 404 for not found, 500 for server errors).
d. Use Express middleware, including at least one custom middleware.
e. Implement JWT-based authentication and protected routes.
f. Use express.json(), express.static() and appropriate middleware such as CORS/logging along with middleware chaining.
g. Handle API errors appropriately.
h. Store sensitive information such as database credentials and API keys in .env; add .env to .gitignore.
i. Organize the backend into appropriate modules such as routes, controllers, models, and middleware.
j. Define schemas in the /models directory.
k. Save persistent data using MongoDB and Mongoose
l. Use CRUD (atleast one each) operations.
m. Use express.static() and express.json() middleware
n. Write atleast one custom middleware function and use the middleware stack/chaining
o. Store sensitive data (API keys, database URIs) in environment variables using a .env file, and add this file to .gitignore.

D. Code Quality, Project Structure and Testing

a. Test all major API endpoints using Postman.
b. Show successful and unsuccessful requests.
c. Test authentication and invalid inputs

E. Documentation & Deployment

a. Follow a clear and modular folder structure.
b. Follow ES6+ standards.
c. Prefer const/let and arrow functions where appropriate.
d. Follow consistent naming conventions:
    i. camelCase for variables/functions
    ii. PascalCase for React components
e. Avoid unnecessary code duplication and create reusable components/modules.
f. Maintain the project on GitHub.
g. Include a README.md containing:
    i. Project description and features
    ii. Installation and execution instructions
    iii. Project structure
    iv. API endpoints
    v. Environment-variable requirements
h. Submit a short report containing screenshots and major functionality.
i. Deploy the application and provide the working URL.